

# **COS214 Prac3**

U25054245 - Nathan Spavins

U25420314 - Christian Khaled

U25265718 - Jack Reid

---

## **Section 1:**

**1.1.** Our event is a marathon. It can be used by any organiser who is running a marathon event. A marathon stretches for kilometers, has many runners, and has many conditions that change in the moment. Five concrete kinds of event units we have included is a water table that give water bottles to runners and can run out of stock, medical tent that tends to injured runners and carries on working even if everything else around closes, spectator stands full of people watching the marathon that has a limit and can be cleared if closed, marshal point where there is a marshal that helps and keeps the runners on track or warns them of anything ahead, they become more active during incidents and timing mat to record a runners time when a runner runs over it even if the section it is in is closed. These are all units in this marathon that are spread out along the marathon route and need to have announcements telling them to change for conditions like weather, bad road, full capacity, etc. There can be three kinds of grouping areas that it could be, it could be a central hub, maybe by a town, that has medical tents, water tables, and even in some scenarios could have spectator stands. It could be a actual part of the course like a road, this will have marshal points, and/or timing mats. It could also be the start or finish of the race this will contain medical tents, timing mats and spectator stands. Some of these areas could be nested into others for example multiple central hubs of water tables and medical tents could be nested in a very long road section.

## **1.2. Observer:**

Subject: Subject

ConcreteSubject: Control, CourseSection

Observer: Observer

ConcreteObserver: CourseSection, WaterTable, MedicalTent,  
SpectatorStands, MarshalPoint, TimingMat

## Composite:

Component: MarathonEvent

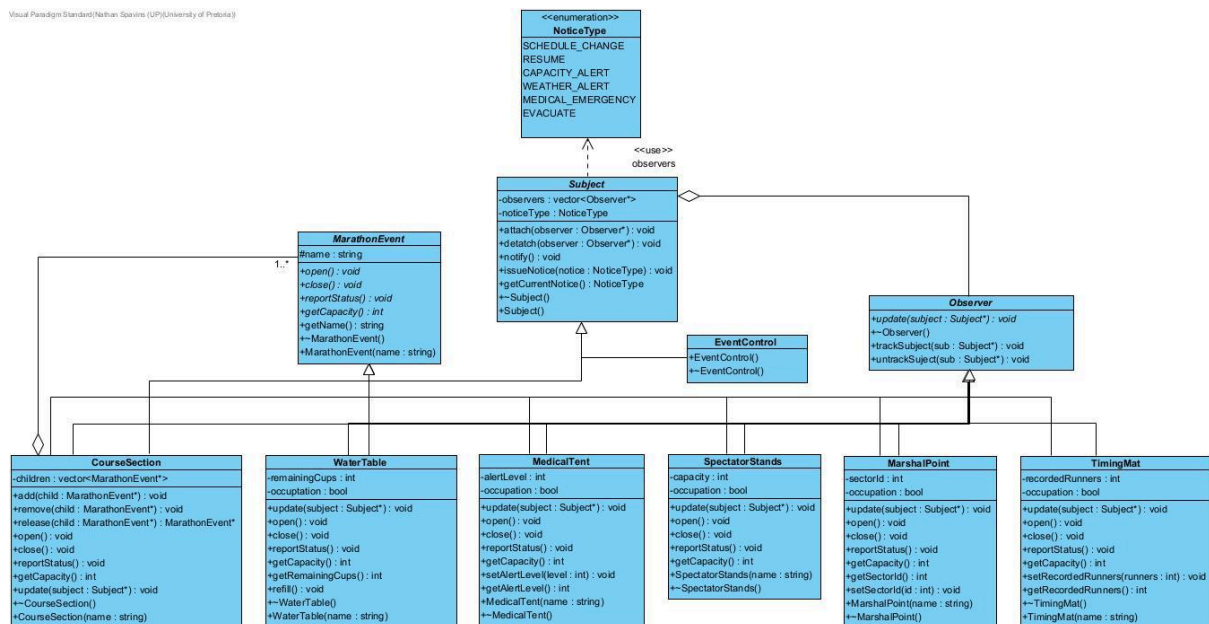
Leaf: WaterTable, MedicalTent, SpectatorStands, MarshalPoint, TimingMat

Composite: CourseSection

The composite and leaves of the composite pattern are also concrete observers. The reason why is the RaceControl will notify the observers to change and all the different components of the marathon need to change if something happens like for example weather. This means that they need to be concrete observers as they take the interface the abstract classes defined and implements it differently depending on the concrete observer, like a CourseSection will need to update its children as well, where as a water table will close if there is no more water. They are also leaves and the composite in the composite pattern however as we need to compose the objects into a tree structure to the marathon a hierarchy of components building the event, which is why the CourseSection, WaterTable, MedicalTent, SpectatorStands, MarshalPoint, TimingMat are the leaves/Composite(CourseSection) of the Composite pattern, as well as concrete Observers of the Observer pattern. The Composite (CourseSection) is also a concrete subject as it has to notify its children.

### 1.3.

Visual Paradigm Standard/Nathan Sparrow (UP)University of Pretoria



### 1.4.

- The composite is a genuine whole tree as **MarathonEvent** is the common abstract component, **CourseSection** is the composite and **TimingMat**, **WaterTable**, **MedicalTent**, **MarshallPoint** are the concrete leaves. A **CourseSection** can contain multiple **MarathonEvent** components and can also contain other **courseSection** objects which create nested branches and with leaves can be attached. This along with operations such as **open()**, **close()**,

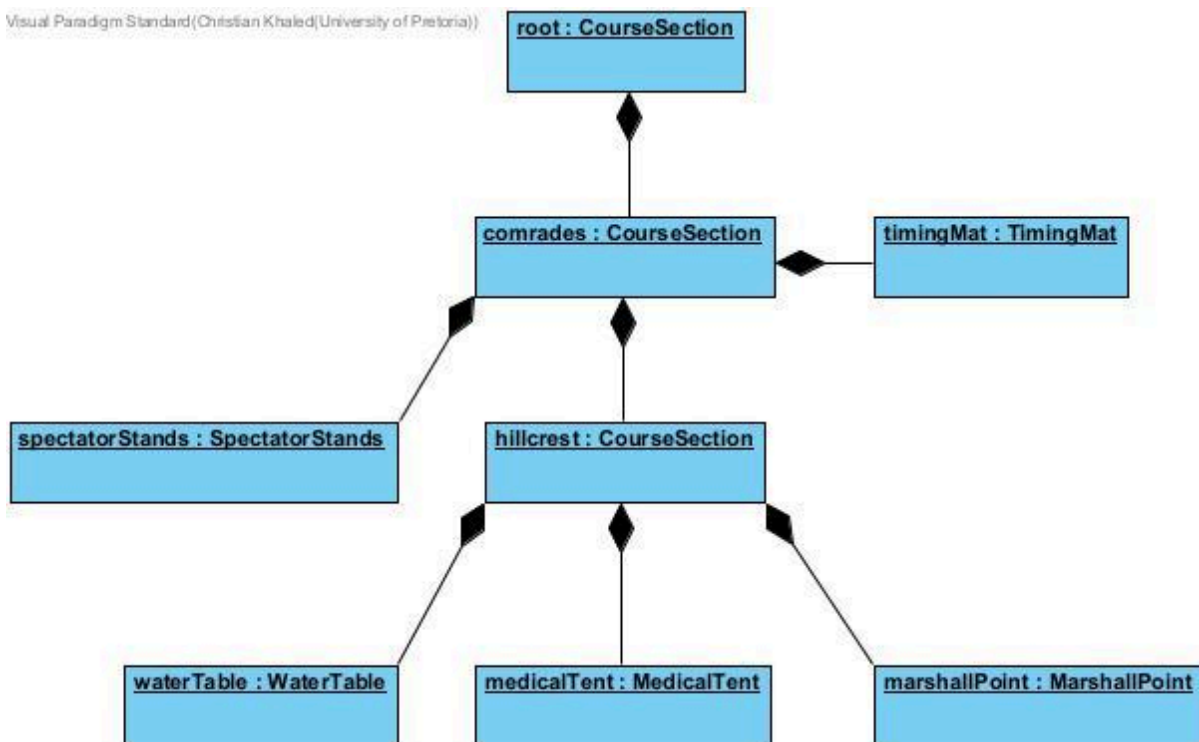
getCapacity() and reportStatus can therefore be labelled as a genuine part-whole Tree.

- b) The observer Pattern is required because notices such as evacuations and weather alerts may affect multiple independent event components. Hard coding calls from the Subject directly to each concrete event would create strong coupling between the Subject and every type of event that needs to react. Thus this is why an observer is better than direct hard-coded calls.
- c) The ownership of event components is determined by composite relationship. The component responsible for managing the composites children is responsible for their lifetime. In this case it would be the CourseSection that owns its child components once they are added to it and is responsible for managing their lifetime.
- d) Subjects and Observers don't own the objects they reference. A subject stores non owning pointers to its Observers, while an Observer stores non-owning pointers to its Subjects. The objects are owned and destroyed by whoever created them and references must be removed before the object is destroyed to avoid dangling pointers.

## Section 2:

### 2.3

Visual Paradigm Standard (Christian Khaled (University of Pretoria))



**2.4** For the composite Destruction, CourseSections destructor iterates its children vector and calls delete on each owned pointer, it makes use of recursion to do the same behaviour in nested composites. Since each child is owned by exactly one parent's child vector, deleting the root cascades through the entire tree exactly once.

This means no leaks and no double deletion. For the transfer of ownership, when a unit moves between composite, it releases the child, which essentially removes the pointer without deleting it. This ensures that an object is never owned by 2 or more composites at once, preventing the idea of a double free.

### **Section 3:**

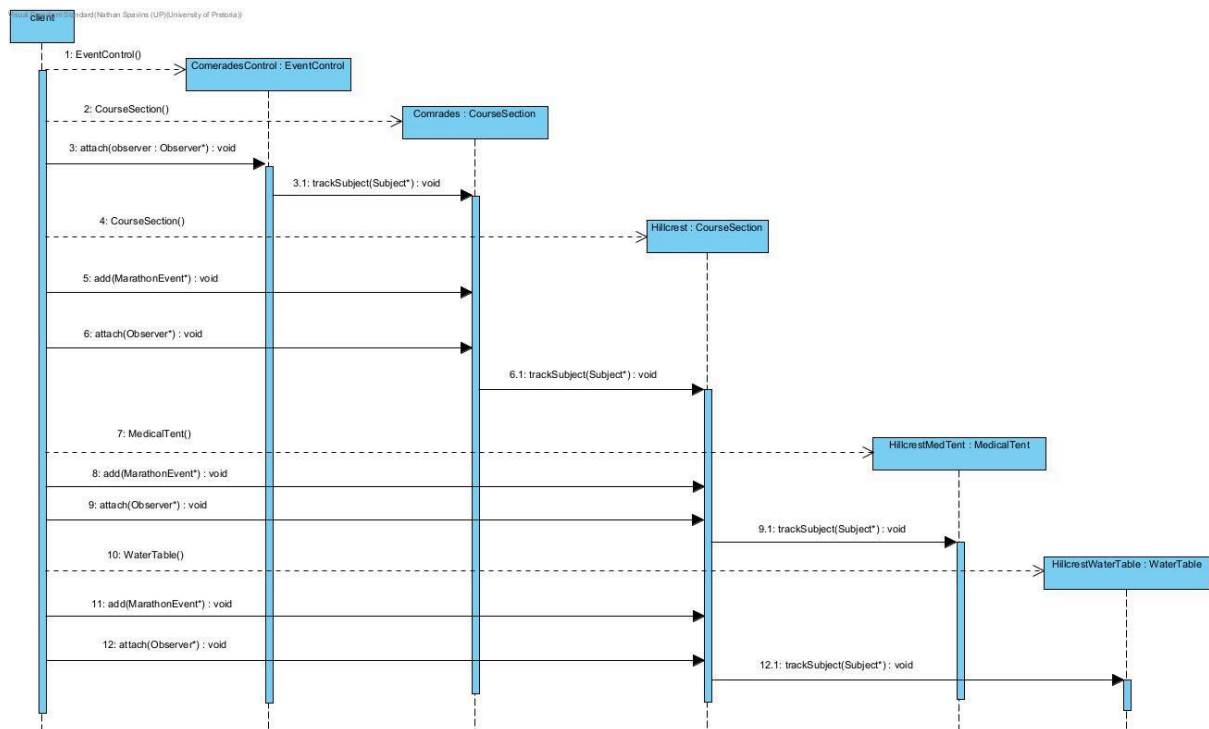
**3.1** The policy is that the duplicate attach() is ignored. Before we add an observer, attach() checks via find whether or not the point already exists in observers. This will prevent the same observer receiving multiple update calls per a notify, which would typically call unintended duplicates. For detach() on an unregistered observer. We use the erase-remove concept. If a pointer isn't found, nothing is removed and no error will occur. This makes detach self-consistent which is required by a self detaching destructor policy.

**3.2** We decided that we want our observers to detach themselves. This will guarantee lifetime ordering because each observer will bind its deregistration logic, directly to their own destruction lifestyle. This is because to solve the general problem where an observable object keeps a dead or dangling observer alive inside of memory and will attempt to notify an invalid memory address. This is why we choose self-detatching observers

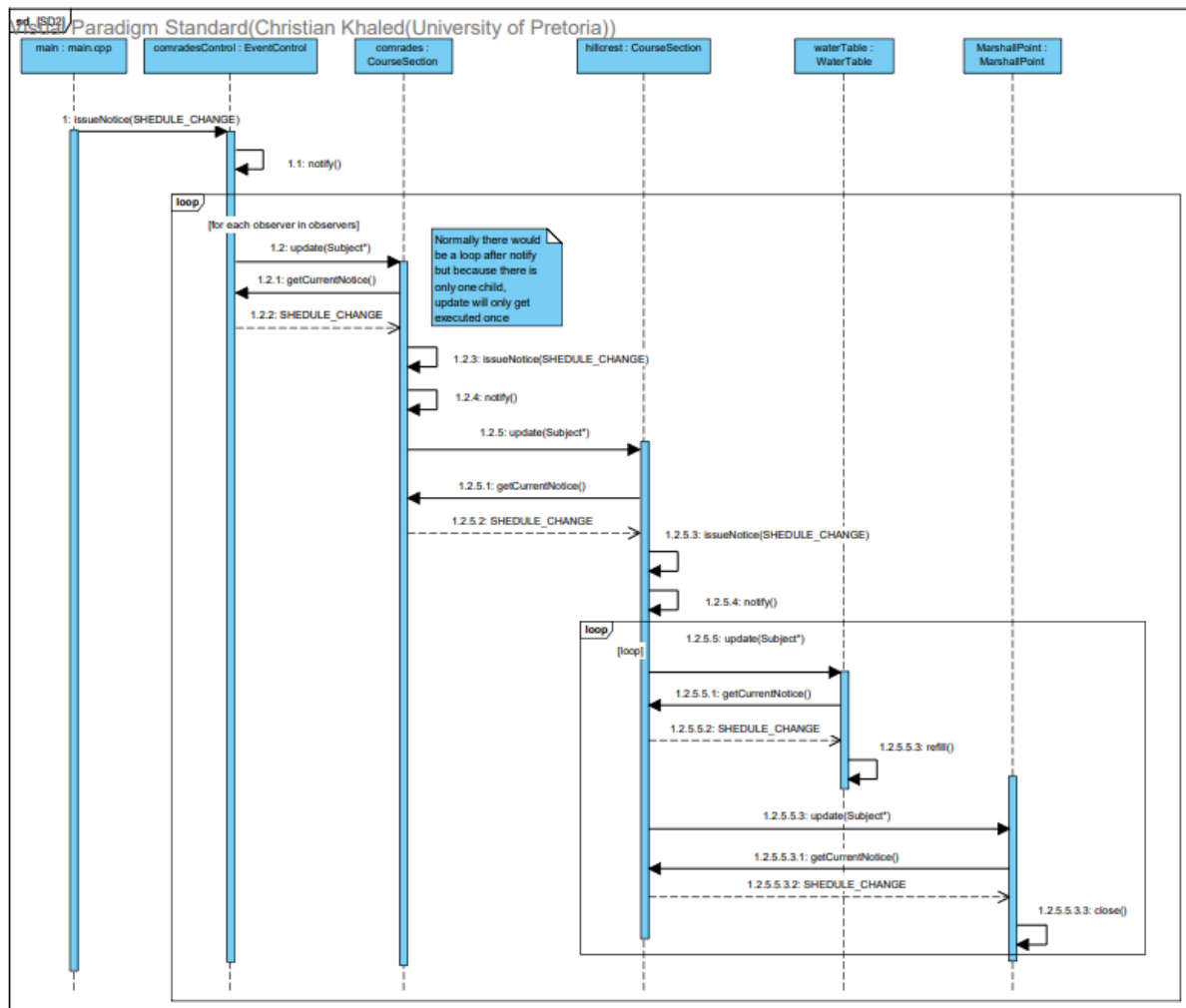
**3.5** We decided to use the pull implementation. notify() calls update(Subject\*) on each observer with no event data attached. Observer only has a reference to the subject that has been changed and needs to call on itself with getCurrentNotice to get whatever state it is looking for. Pull only uses a subject\* reference and the update stays generic which means it works for any Subject subclass. In our design we have multiple concrete subjects (EventControl and CourseSection) that both notify the same Observer interface. Pull keeps any update function decoupled from any specific or hard coded notice implementations.

### **Section 5:**

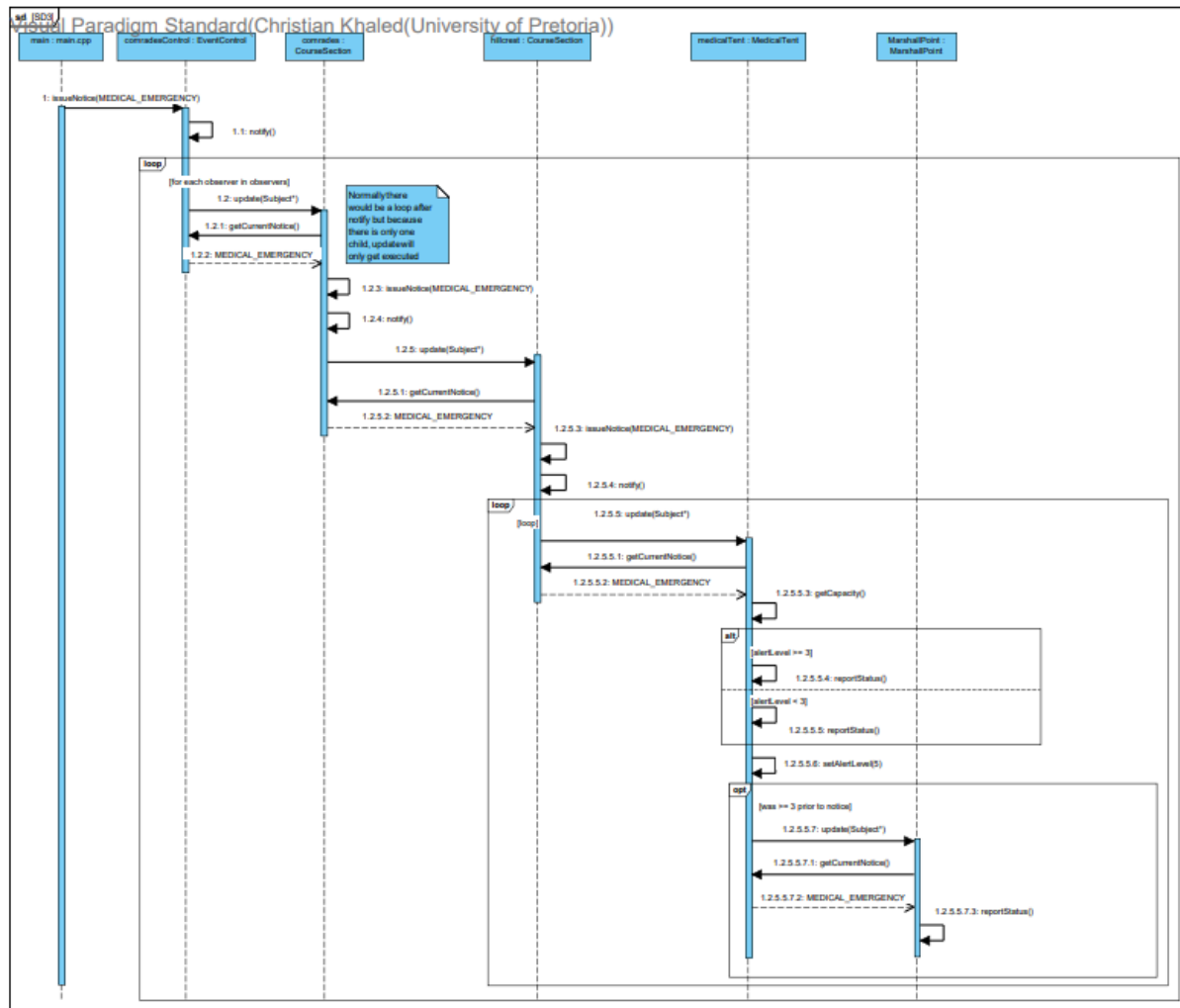
## 5.1



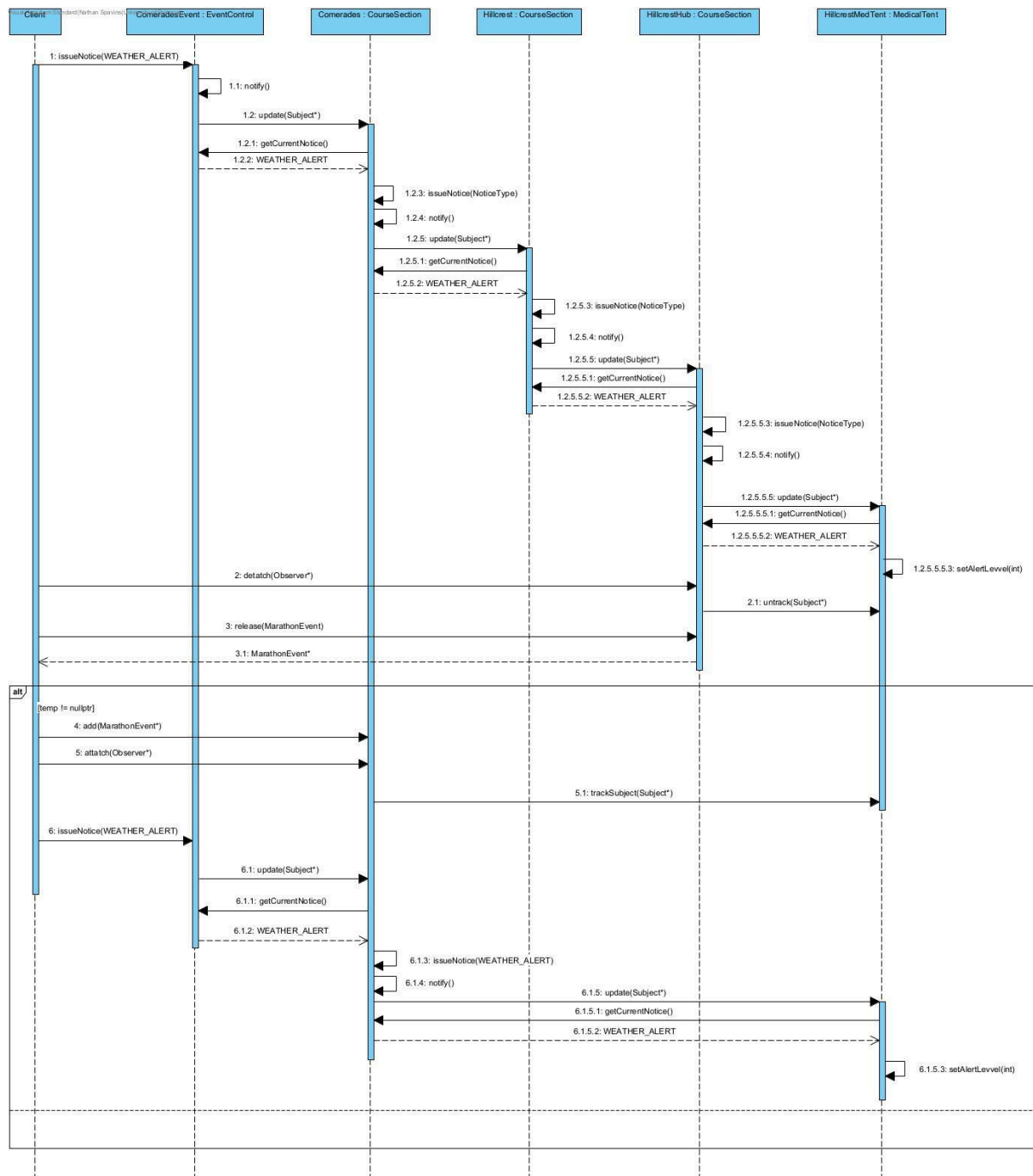
## 5.2



## 5.3



## 5.4



## Section 7:

**7.4** One of the members created the git repository and sent it to all of us via email. After creating he then focused on completing the UML and a couple of theory questions while the other 2 worked on the code. One person focused on the observer and when they finished basic implementation they added a commit. The other person focused on composite and once completed added that to the repository too. We then pulled the combined information and split up the 5 leafs between the 2 that were doing the initial coding. We then committed and pushed our updated and



completed leaves. Now that the majority of the code was done the first group member worked on the main and makefile the other 2 started with sequence diagrams and more theory questions. The main and make file was then committed to the repository. We made use of pull and push requests alongside a VS code extension so we could access and update the code directly from our terminal and not have to go to the actual GitHub website.